

Module – 3:
Introduction To Raspberry Pi
Pico

Introduction – Raspberry Pi Pico

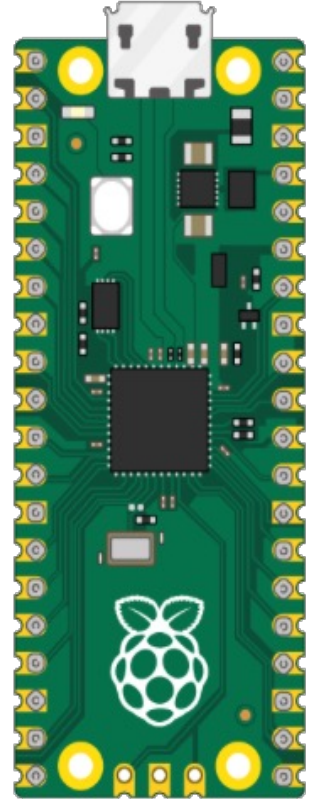
- Raspberry Pi Pico is a microcontroller board developed by the Raspberry Pi Foundation
- Raspberry Pi Pico is a “downscaled” version of the original Raspberry Pi and is more comparable with Arduino compared to the original Raspberry Pi

We have 4 different types:

- **Raspberry Pi Pico** (original)
- **Raspberry Pi Pico H** - pre-soldered header pins included
- **Raspberry Pi Pico W** – WiFi included
- **Raspberry Pi Pico WH** – WiFi and pre- soldered header pins included

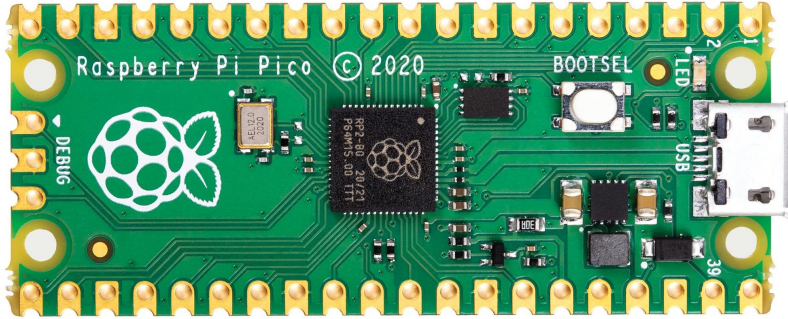
<https://www.raspberrypi.com/products/raspberry-pi-pico/>

<https://projects.raspberrypi.org/en/projects/getting-started-with-the-pico>

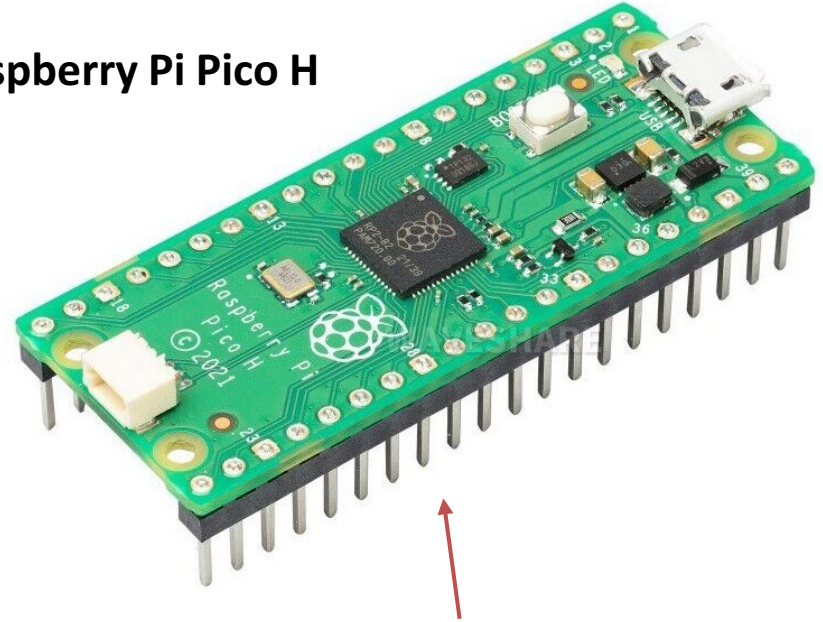


Raspberry Pi Pico Series

Raspberry Pi Pico (original)

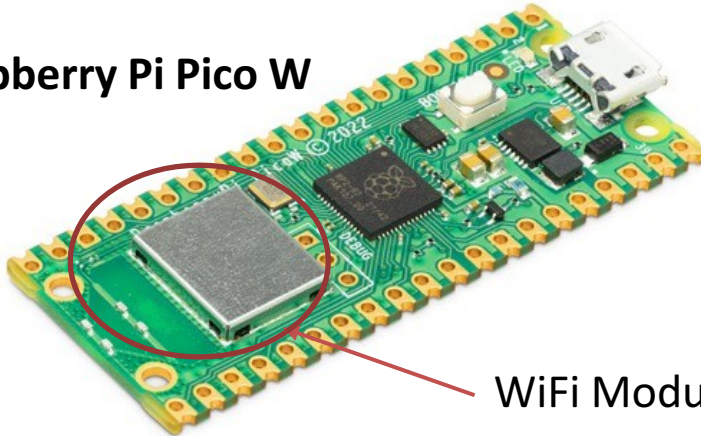


Raspberry Pi Pico H



Pre-soldered header pins included

Raspberry Pi Pico W

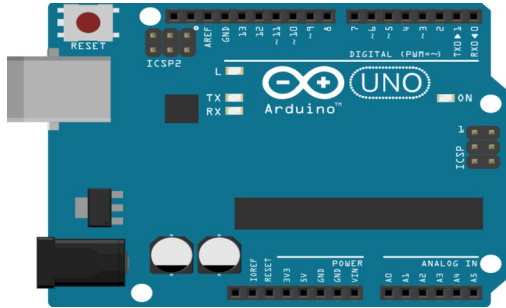


WiFi Module and Antenna

Arduino vs. Raspberry Pi

Arduino Family

Arduino UNO



Arduino UNO and similar Arduino boards is a Microcontroller Unit (MCU)

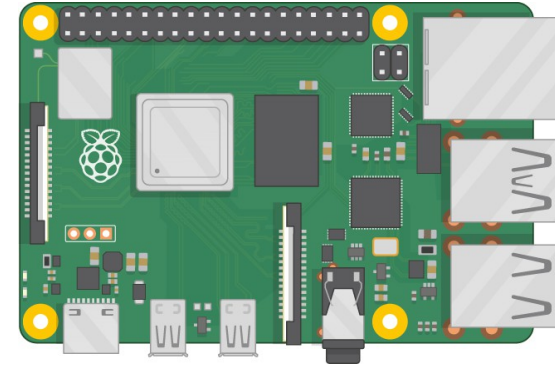
Programming Language:
Arduino IDE and C/C++

Raspberry Pi is a Single-Board Computer (SBC), which is a microcontroller unit with CPU, RAM, and external hard disk.

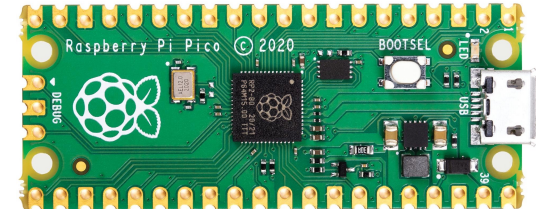
Operating System: Linux
Programming Language:

Python + many others
Raspberry Pi Pico is a Microcontroller Unit (MCU)
Programming Language:
MicroPython or C/C++

Raspberry Pi



Raspberry Pi Pico

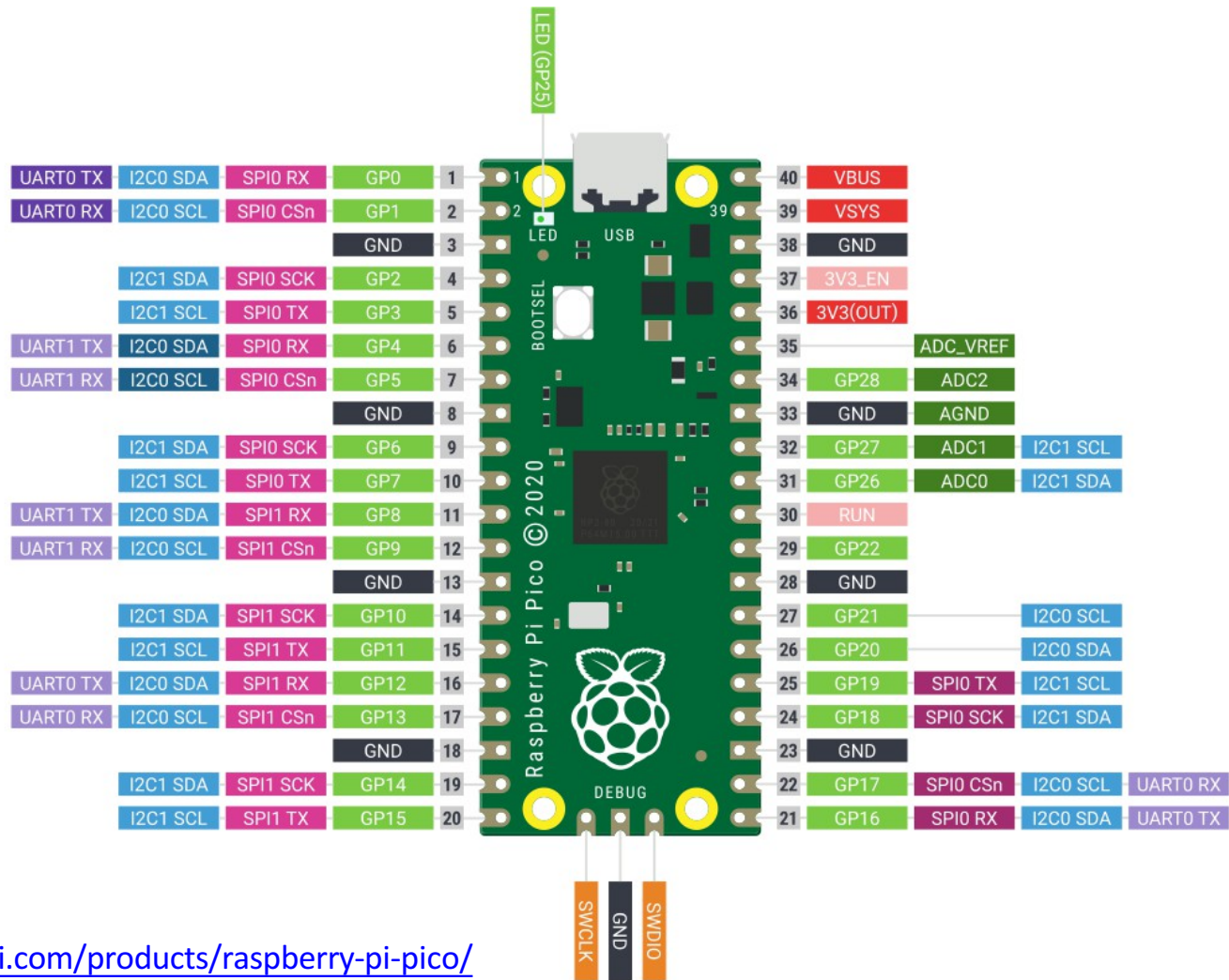


Raspberry Pi Pico Specifications

- Size: 21 mm × 51 mm
- **Micro-USB B port** for power and data
- CPU: Dual-core Arm Cortex-M0+ @ 133MHz
- Memory: 264KB on-chip SRAM; 2MB onboard QSPI Flash
- Interface: **26 GPIO pins**, including **3 Analog Inputs (ADC)**
- Peripherals:
 - 2 × UART
 - 2 × **SPI** controllers
 - 2 × **I2C** controllers
 - 16 × **PWM** channels

Pico Pinout

■	Power
■	Ground
■	UART / UART (default)
■	GPIO, PIO, and PWM
■	ADC
■	SPI / SPI (default)
■	I2C / I2C (default)
■	System Control
■	Debugging



<https://www.raspberrypi.com/products/raspberry-pi-pico/>

Pico Pinout

The 40 pins on a [Raspberry Pi Pico](https://www.raspberrypi.com/products/raspberry-pi-pico/) are organized into functional categories to handle power, general-purpose tasks, and specialized communication protocols.

1. Power & Ground Pins : These pins provide electrical power to the board and external components.

- **VBUS (Pin 40):** Provides 5V directly from the micro-USB port.
- **VSYS (Pin 39):** The main system input (1.8V to 5.5V). Use this to power the Pico from batteries or external power supplies.
- **3V3 (Pin 36):** Regulated 3.3V output from the onboard converter. Use this to power external sensors (max ~300mA).
- **GND (8 Pins):** Common ground points located at pins 3, 8, 13, 18, 23, 28, 33, and 38.

2. General-Purpose I/O (GPIO) Pins: The Pico has **26 multi-function GPIO pins** (GP0–GP22, GP26–GP28).

- **Digital I/O:** All GPIOs can be used to send (output) or receive (input) high/low signals (3.3V logic).
- **PWM (Pulse Width Modulation):** All GPIO pins can generate PWM signals, useful for dimming LEDs or controlling motor speed. There are 16 independent PWM channels available.

3. Analog-to-Digital (ADC) Pins : These pins read varying voltage levels (analog) and convert them to digital values.

- **ADC0, ADC1, ADC2 (Pins 31, 32, 34):** Externally accessible pins mapped to GP26, GP27, and GP28.
- **ADC3 (Internal):** Used for monitoring the VSYS voltage.
- **ADC4 (Internal):** Connected to an onboard temperature sensor.
- **Resolution:** Features 12-bit hardware resolution (0–4095), often scaled to 16-bit (0–65535) in Micro-Python.

4. Communication Interface Pins : Most GPIO pins can be reconfigured to handle specific serial protocols:

- **UART (2 controllers):** Standard serial communication (TX/RX) for connecting to GPS, Bluetooth, or computers.
- **I2C (2 controllers):** Two-wire interface (SDA/SCL) for sensors and OLED displays.
- **SPI (2 controllers):** High-speed interface (MOSI/MISO/SCK/CS) for SD cards and TFT screens.

5. System Control & Debug Pins

- **RUN (Pin 30):** Master reset pin. Pulling it LOW restarts the Pico.
- **3V3_EN (Pin 37):** Disables the 3.3V power regulator when pulled LOW.
- **SWD Debug Port:** Three pins at the bottom edge (SWDIO, GND, SWCLK) used for advanced hardware debugging.

<https://www.raspberrypi.com/products/raspberry-pi-pico/>

Raspberry Pi Pico - Programming

Raspberry Pi Pico programming typically involves using **MicroPython** for ease of use or **C/C++** for high-performance applications. You program it by connecting it to a computer via USB and "flashing" code directly onto its onboard flash memory.

Programming Languages

- **MicroPython:** The most popular and beginner-friendly option. It allows you to write Python code that runs directly on the Pico's hardware.
- **CircuitPython:** A version of MicroPython created by Adafruit, designed to be even simpler with extensive library support for many sensors.
- **C/C++:** Used for maximum performance and efficiency. It requires the official **Pico SDK** and a compilation step before flashing.
- **Arduino C++:** You can use the familiar Arduino IDE and its vast library ecosystem to program the Pico.

Recommended Tools (IDEs)

- **Thonny IDE:** The official recommendation for MicroPython. It is lightweight and includes built-in tools to install firmware and manage files on the Pico.
- **Visual Studio Code (VS Code):** Recommended for C/C++ development. An official **Raspberry Pi Pico extension** simplifies project setup, toolchain management, and debugging.
- **Arduino IDE:** A great choice for those transitioning from the Arduino ecosystem who want to use C++.

Raspberry Pi Pico - Programming

Basic Workflow (MicroPython)

1. **Enter BOOTSEL mode:** Hold the **BOOTSEL** button while plugging the Pico into your computer via USB.
2. **Flash Firmware:** Drag and drop a **UF2** firmware file (downloaded from the [official site](#)) onto the Pico, which appears as a USB drive named RPI-RP2.
3. **Write Code:** Open Thonny, select the "MicroPython (Raspberry Pi Pico)" interpreter, and start coding.
4. **Save and Run:** Save your script as `main.py` directly to the Pico to make it run automatically whenever the board is powered.

Flash firmware → Write code → Run → Save as `main.py` → Auto-run

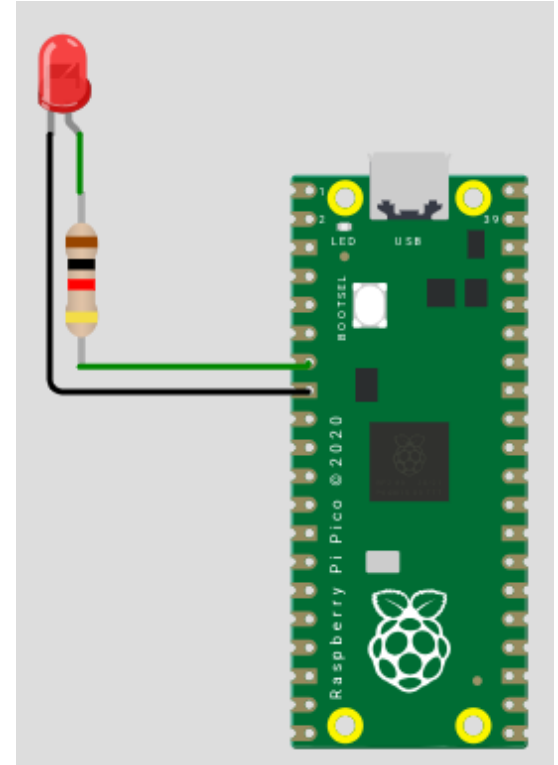
Basic Workflow (C/C++)

1. **Install Pico SDK & Toolchain :** Set up ARM GCC, CMake, and Raspberry Pi Pico SDK.
2. **Write Program :** Create a C/C++ file (`main.c`) using Pico SDK functions.
3. **Configure Build :** Use **CMake** to generate build files.
4. **Compile Code :** Build the project to generate a **.uf2** file.
5. **Upload & Run :** Copy `.uf2` to Pico in **BOOTSEL mode** → program runs automatically.

Write C/C++ → Configure with CMake → Compile → Generate UF2 → Flash → Run

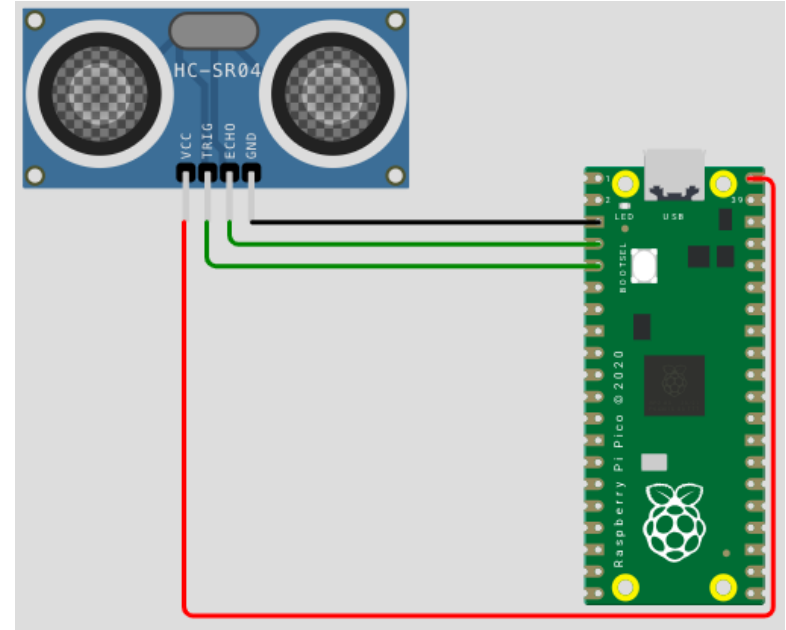
Example-1 : LED Blinking using Pico

```
1  #include "pico/stdlib.h"
2
3  #define LED_PIN 5
4
5  int main() {
6      gpio_init(LED_PIN);
7      gpio_set_dir(LED_PIN, GPIO_OUT);
8
9      while (true) {
10         gpio_put(LED_PIN, 1);
11         sleep_ms(500);
12         gpio_put(LED_PIN, 0);
13         sleep_ms(500);
14     }
15 }
```



Example-2 : Ultrasonic Sensor with Pico

```
1  #include <stdio.h>
2  #include "pico/stdlib.h"
3
4  #define TRIG_PIN 3
5  #define ECHO_PIN 2
6
7  int main() {
8      stdio_init_all();
9
10     gpio_init(TRIG_PIN);
11     gpio_set_dir(TRIG_PIN, GPIO_OUT);
12
13     gpio_init(ECHO_PIN);
14     gpio_set_dir(ECHO_PIN, GPIO_IN);
15
16     sleep_ms(2000);
17
18     while (true) {
19         gpio_put(TRIG_PIN, 0);
20         sleep_us(2);
21
22         gpio_put(TRIG_PIN, 1);
23         sleep_us(10);
24         gpio_put(TRIG_PIN, 0);
25
26         while (gpio_get(ECHO_PIN) == 0);
27         absolute_time_t start = get_absolute_time();
28
29         while (gpio_get(ECHO_PIN) == 1);
30         absolute_time_t end = get_absolute_time();
31     }
```

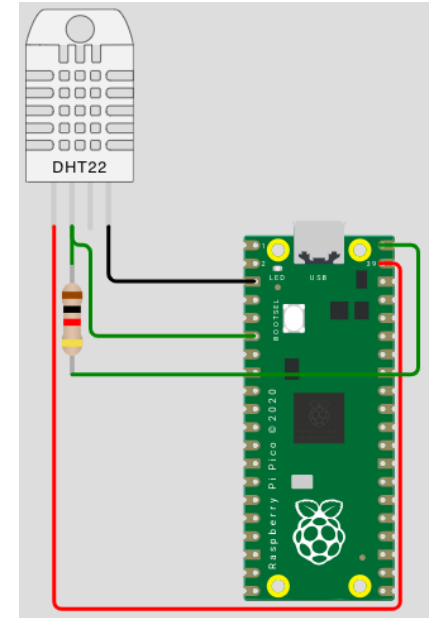
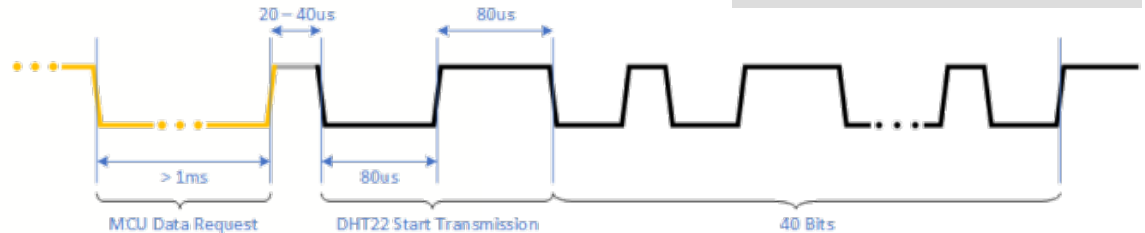


```
32     int64_t duration = absolute_time_diff_us(start, end);
33
34     float distance_cm = duration * 0.0343 / 2;
35
36     printf("Distance: %.2f cm\n", distance_cm);
37
38     sleep_ms(500);
39 }
40
```

<https://wokwi.com/projects/456860437384802305>

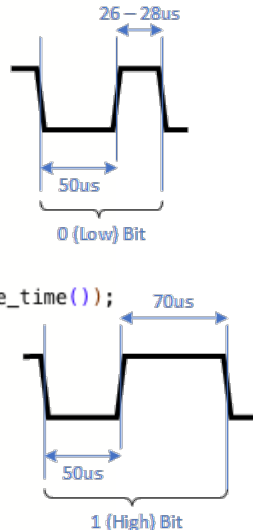
Example-3 : DHT22 with R-Pi Pico

```
1  #include <stdio.h>
2  #include "pico/stdlib.h"
3
4  #define DHT_PIN 4
5
6  uint8_t dht[5];
7
8  bool wait_level(bool level, int timeout_us) {
9      absolute_time_t start = get_absolute_time();
10     while (gpio_get(DHT_PIN) != level) {
11         if (absolute_time_diff_us(start, get_absolute_time()) > timeout_us)
12             return false;
13     }
14     return true;
15 }
16
17 void dht_start(void) {
18     gpio_set_dir(DHT_PIN, GPIO_OUT);
19     gpio_put(DHT_PIN, 0);
20     sleep_ms(2);
21     gpio_put(DHT_PIN, 1);
22     sleep_us(30);
23     gpio_set_dir(DHT_PIN, GPIO_IN);
24 }
```



Example-3 : DHT22 with R-Pi Pico

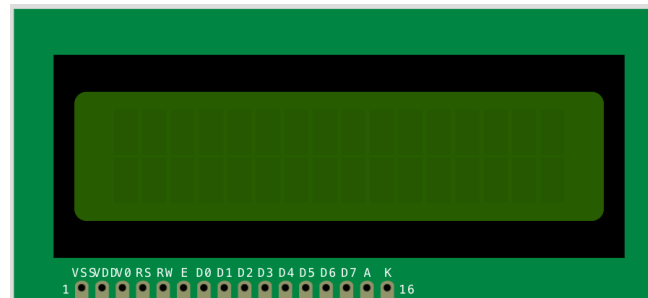
```
26 bool dht_read_bits(void) {
27     for (int i = 0; i < 5; i++) dht[i] = 0;
28
29     if (!wait_level(0, 100)) return false;
30     if (!wait_level(1, 100)) return false;
31     if (!wait_level(0, 100)) return false;
32
33     for (int i = 0; i < 40; i++) {
34         if (!wait_level(1, 100)) return false;
35         absolute_time_t start = get_absolute_time();
36         if (!wait_level(0, 100)) return false;
37
38         int t = absolute_time_diff_us(start, get_absolute_time());
39         dht[i / 8] <= 1;
40         if (t > 40) dht[i / 8] |= 1;
41     }
42     return true;
43 }
44
45 bool dht_print(void) {
46     uint8_t sum = dht[0] + dht[1] + dht[2] + dht[3];
47     if ((sum & 0xFF) != dht[4]) return false;
48
49     int16_t raw_h = (dht[0] << 8) | dht[1];
50     int16_t raw_t = (dht[2] << 8) | dht[3];
51
52     float humidity = raw_h / 10.0f;
53     float temperature = (raw_t & 0x8000)
54         ? -(raw_t & 0x7FFF) / 10.0f
55         : raw_t / 10.0f;
```



```
57     printf("Humidity: %.1f %%\n", humidity);
58     printf("Temperature: %.1f C\n\n", temperature);
59     return true;
60 }
61
62 void dht_bus_reset(void) {
63     gpio_set_dir(DHT_PIN, GPIO_OUT);
64     gpio_put(DHT_PIN, 1);
65     sleep_ms(10);
66     gpio_set_dir(DHT_PIN, GPIO_IN);
67 }
68
69 int main() {
70     stdio_init_all();
71     sleep_ms(2000);
72
73     gpio_init(DHT_PIN);
74     gpio_pull_up(DHT_PIN);
75
76     while (true) {
77         printf("Starting DHT read\n");
78
79         dht_start();
80
81         if (dht_read_bits()) {
82             if (!dht_print())
83                 printf("Checksum error\n\n");
84         } else {
85             printf("Sensor timeout\n\n");
86         }
87
88         dht_bus_reset();
89         sleep_ms(2500);
90     }
91 }
```

Example-4 : LCD Interfacing with PICO

```
1  #include "pico/stdlib.h"
2  #include <stdbool.h>
3
4  // --- Pin definitions ---
5  #define RS 2
6  #define E 3
7  #define D4 4
8  #define D5 5
9  #define D6 6
10 #define D7 7
11
12 // --- Helper functions ---
13
14 void pulse_enable() {
15     gpio_put(E, 0); sleep_us(1);
16     gpio_put(E, 1); sleep_us(1);
17     gpio_put(E, 0); sleep_us(100);
18 }
19
20 void send_nibble(uint8_t nib) {
21     gpio_put(D4, (nib >> 0) & 1);
22     gpio_put(D5, (nib >> 1) & 1);
23     gpio_put(D6, (nib >> 2) & 1);
24     gpio_put(D7, (nib >> 3) & 1);
25     pulse_enable();
26 }
27
28 void send_byte(uint8_t byte, bool is_data) {
29     gpio_put(RS, is_data);
30     send_nibble(byte >> 4); // high nibble
31     send_nibble(byte & 0x0F); // low nibble
32     sleep_ms(2);
33 }
34
35 void lcd_print(const char* str) {
36     while (*str) send_byte(*str++, true);
37 }
38
39 void lcd_set_cursor(int col, int row) {
40     int row_offsets[] = {0x00, 0x40};
41     send_byte(0x80 + col + row_offsets[row], false);
42 }
43
44 void lcd_clear() {
45     send_byte(0x01, false);
46     sleep_ms(2);
47 }
48
49 void lcd_init() {
50     // Initialize GPIO pins
51     uint pins[] = {RS, E, D4, D5, D6, D7};
52     for (int i = 0; i < 6; i++) {
53         gpio_init(pins[i]);
54         gpio_set_dir(pins[i], true);
55     }
56
57     sleep_ms(50); // power-on delay
58
59     // 4-bit initialization sequence
60     send_nibble(0x03); sleep_ms(5);
61     send_nibble(0x03); sleep_us(150);
62     send_nibble(0x03);
63     send_nibble(0x02);
64
65     send_byte(0x28, false); // 4-bit, 2 lines, 5x8
66     send_byte(0x0C, false); // display on, cursor off
67     send_byte(0x06, false); // entry mode set
68     lcd_clear();
69 }
```



LCD Pin	Function	Connect To Pico Pin / Notes
VSS	Ground	GND
VDD	Power	5V
V0	Contrast	Middle pin of 10 kΩ potentiometer (ends to 5V and GND)
RS	Register Select	GPIO 2
RW	Read/Write	GND (always write)
E	Enable	GPIO 3
D4	Data Bit 4	GPIO 4
D5	Data Bit 5	GPIO 5
D6	Data Bit 6	GPIO 6
D7	Data Bit 7	GPIO 7
A (LED+)	LCD Backlight Anode	5V through 220 Ω resistor
K (LED-)	LCD Backlight Cathode	GND

Example-4 : LCD Interfacing with PICO

```
35 void lcd_print(const char* str) {
36     while (*str) send_byte(*str++, true);
37 }
38
39 void lcd_set_cursor(int col, int row) {
40     int row_offsets[] = {0x00, 0x40};
41     send_byte(0x80 + col + row_offsets[row], false);
42 }
43
44 void lcd_clear() {
45     send_byte(0x01, false);
46     sleep_ms(2);
47 }
48
49 void lcd_init() {
50     // Initialize GPIO pins
51     uint pins[] = {RS, E, D4, D5, D6, D7};
52     for (int i = 0; i < 6; i++) {
53         gpio_init(pins[i]);
54         gpio_set_dir(pins[i], true);
55     }
56
57     sleep_ms(50); // power-on delay
58
59     // 4-bit initialization sequence
60     send_nibble(0x03); sleep_ms(5);
61     send_nibble(0x03); sleep_us(150);
62     send_nibble(0x03);
63     send_nibble(0x02);
64
65     send_byte(0x28, false); // 4-bit, 2 lines, 5x8
66     send_byte(0x0C, false); // display on, cursor off
67     send_byte(0x06, false); // entry mode set
68     lcd_clear();
69 }
```

```
72 void bin4(uint8_t num, char* out) {
73     int n = num;
74     for(int i = 3; i >= 0; i--) {
75         out[i] = (n % 2) ? '1' : '0';
76         n = n / 2;
77     }
78     out[4] = '\0';
79 }
80
81 // --- Main program ---
82 int main() {
83     stdio_init_all();
84     lcd_init();
85
86     char buf[5];
87     uint8_t counter = 0;
88
89     while (1) {
90         bin4(counter, buf);
91         lcd_set_cursor(0, 0);
92         lcd_print("Binary: ");
93         lcd_print(buf);
94
95         counter = (counter + 1) & 0x0F;
96         sleep_ms(500);
97     }
98 }
```

